

**PENGUJIAN PERANGKAT LUNAK
(TIFNJK140705)**

SEMESTER IV



TEST PLAN “Zoovia”

Oleh :

Adnan Tri Handoko	(E41230684)
Dwisna Risma Nur Hamida	(E41230674)
Tengku Farkhan Arsyad S	(E41230747)
Shava Selvia Ramadhani S	(E41231350)
Candra Puji Utama	(E41231578)

**PROGRAM STUDI TEKNIK INFORMATIKA KAMPUS 3 NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2025**

DAFTAR ISI

Cover	1
1. Overview	3
2. Bounds	4
3. Quality Risk	6
4. Proposed Schedule of Milestones	8
5. Transition	9
5.1 Entry Criteria	9
5.2 Continuation Criteria	9
5.3 Exit Criteria	9
6. Test Development	10
6.1 Pembuatan dan Pengelolaan Dokumen Pengujian	10
6.2 Metodologi/Strategi Pengujian	11
6.3 Jenis Pengujian yang dilakukan	11
7. Test Configuration and environment	14
7.2 Software Uji	15
A. Tools yang Digunakan dalam Pengujian:	15
7.3 Konfigurasi Lingkungan Pengujian	16
A. Koneksi Internet:	16
B. Database:	16
C. Akun Pengujian:	16
D. Skenario Pengujian:	16
8. Test Execution	17
8.1 Resources	17
8.2 Test Case and Bug Tracking	17
8.3 Bug Isolation	18
8.4 Test Release Management	18
8.5 Test Cycles	18
8.6 Test Hours	19
9. Risk and Contingency	20
9.1 Identifikasi Risiko	20
9.2 Rencana Kontingensi	20
10. Change History	22
11. Reference document	23
12. FAQ	24

1. Overview

Project Zoovia adalah project yang bertujuan untuk mengembangkan perangkat lunak berbasis web dan mobile guna mendukung proses manajemen dan pelayanan puskesmas seperti mengelola antrian, mengelola rekam medis hewan, digitalisasi pelayanan, dan juga memberikan informasi kepada masyarakat terkait masalah dan kesehatan hewan. Project Zoovia dibuat berdasarkan rumusan masalah yang telah kami peroleh dari hasil wawancara kami dengan *stakeholder*. Sistem ini bertujuan menjadi solusi digital yang mengintegrasikan berbagai aspek operasional klinik atau fasilitas veteriner, seperti rekam medis digital, penjadwalan pemeriksaan, mengelola antrian responsif, dan pengaturan hak akses pengguna. Dokumen Test Plan ini disusun sebagai kerangka kerja untuk memastikan bahwa setiap komponen sistem berfungsi sesuai dengan spesifikasi teknis, memenuhi kebutuhan pengguna, serta siap diimplementasikan secara stabil dan aman. Pengujian dalam proyek ini akan dilaksanakan dengan pendekatan terstruktur yang mencakup berbagai dimensi kualitas perangkat lunak. Fokus utamanya adalah memverifikasi kesesuaian fitur-fitur inti dengan kebutuhan bisnis, seperti proses pendaftaran pasien hewan yang efisien, pembaruan rekam medis yang akurat, dan manajemen antrian yang responsif terhadap perubahan antrian. Selain itu, pengujian akan menjamin bahwa sistem mampu menangani lonjakan *traffic* pengguna, menjaga keamanan data sensitif, serta menyediakan antarmuka yang mudah digunakan oleh berbagai peran pengguna, seperti admin dan pasien.

Strategi pengujian dirancang secara holistik, menggabungkan metode functional testing untuk validasi fungsionalitas dasar, integration testing untuk memastikan koherensi antar-modul, dan performance testing guna mengukur ketahanan sistem di bawah beban kerja tinggi. *Usability testing* akan melibatkan pengguna akhir guna mengevaluasi kemudahan navigasi dan kepuasan pengalaman. Lalu untuk aspek keamanan kami memastikan bahwa data pasien akan terenkripsi dan tersimpan dengan baik di database. Proses ini melibatkan kolaborasi erat antara tim pengembang, manajer proyek, dan perwakilan pengguna akhir, khususnya dalam tahap User Acceptance Testing (UAT). Hasil akhir yang diharapkan adalah sistem yang tidak hanya bebas dari cacat kritis tetapi juga memenuhi standar kepuasan pengguna, baik dari segi keandalan, kecepatan, maupun keamanan. Dengan demikian, dokumen ini menjadi fondasi untuk memastikan bahwa Puskesmas dapat dioperasikan secara optimal, mendukung layanan kesehatan hewan yang lebih efektif dan terpercaya.

2. Bounds

Bagian ini menguraikan batasan yang membatasi ruang lingkup pengujian, termasuk fitur yang diuji, yang tidak diuji, istilah teknis relevan, serta konteks proyek yang mempengaruhi strategi pengujian. Batasan dirancang untuk mengatur ekspektasi stakeholder, menghindari ambiguitas, dan memastikan fokus tim pada area kritis yang telah disepakati.

2.1. Konteks Proyek

Proyek Zoovia dikembangkan sebagai platform terpusat untuk manajemen klinik hewan, dengan tujuan utama meningkatkan efisiensi pelayanan kesehatan hewan melalui digitalisasi proses administratif dan medis. Sistem ini diharapkan digunakan oleh multi-role pengguna (admin dan pasien) di lingkungan dengan konektivitas internet terbatas. Pengujian dilakukan dalam waktu 8 minggu dengan sumber daya terbatas, sehingga prioritas diberikan pada fitur inti yang langsung berdampak pada operasional harian.

2.2. Batasan Lingkup Pengujian

Pengujian dalam proyek ini difokuskan pada modul inti yang telah ditetapkan dalam dokumen requirement. Meskipun sistem Zoovia bersifat terintegrasi, terdapat beberapa aspek yang sengaja dikecualikan dari fase pengujian saat ini karena keterbatasan waktu, sumber daya, atau prioritas bisnis. Berikut adalah tabel batasan batasan lingkup pengujian:

Kategori	In-Scope	Out-of-Scope
Fungsionalitas	Pendaftaran pasien, rekam medis, mengelola antrian, mengelola artikel, mengelola pasien	Integrasi dengan sistem pembayaran pihak ketiga, integrasi dengan sistem informasi pemerintah (iSIKHNAS)
Performa / Kinerja	Beban hingga 50 pengguna simultan, responsifitas perubahan data	Uji beban di atas 500 pengguna
Keamanan	Enkripsi data, autentikasi dan otorisasi pengguna.	Penetrasi jaringan infrastruktur server.

2.3. Istilah dan Akronim

Beberapa istilah teknis dan akronim yang digunakan dalam pengujian:

Istilah / Akronim	Definisi
UAT	User Acceptance Testing: Tahap uji oleh pengguna akhir
SIT	System Integration Testing: Uji integrasi antar modul
API	Application Programming Interface: Antarmuka komunikasi sistem
E2E	End-to-End Testing: Uji alur sistem secara lengkap
Backend	Komponen sisi server: Database, API, logika bisnis
Defect	Ketidaksesuaian antara hasil aktual dan ekspektasi

2.4. Asumsi dan Ketergantungan

Proses pengujian mengasumsikan bahwa dokumen persyaratan telah final, lingkungan pengujian siap digunakan, dan tim pengembang dapat merespons temuan bug secara cepat. Ketergantungan utama meliputi ketersediaan sumber daya manusia, alat pengujian, dan koordinasi tim. Berikut adalah tabel yang berisi tentang ketergantungan dan dampak jika tidak terpenuhi:

Aspek	Asumsi/Ketergantungan	Resiko jika tidak terpenuhi
Dokumen Acuan	Requirement telah disetujui dan final	Pengujian tidak selaras dengan kebutuhan
Tim Pengembang	Bug ditindaklanjuti dalam 24-48 jam	Akumulasi <i>defect</i> menghambat <i>progress</i>

3. Quality Risk

Bagian ini mengidentifikasi risiko yang dapat mempengaruhi kualitas sistem Zoovia serta kriteria minimum yang harus dipenuhi untuk memastikan aplikasi layak digunakan. Meskipun aplikasi ini bersifat sederhana, fokus tetap diberikan pada keandalan fungsionalitas inti, keamanan dasar, dan kepuasan pengguna akhir.

3.1. Resiko Kualitas Sistem

Risiko Kualitas Sistem merujuk pada potensi kegagalan atau kelemahan yang dapat mengurangi keandalan, keamanan, atau kepuasan pengguna dalam aplikasi Zoovia. Risiko ini mencakup:

A. Ketidakstabilan Fungsionalitas Inti

Ketidakstabilan Fungsionalitas Inti adalah kondisi di mana fitur-fitur kritis atau esensial dari suatu sistem tidak berjalan secara konsisten, mengalami kegagalan, atau menghasilkan output yang tidak sesuai dengan ekspektasi. Pada proyek Zoovia, fitur inti mencakup modul-modul yang menjadi inti operasional klinik hewan, seperti pendaftaran pasien, penyimpanan rekam medis, dan mengelola antrian. Untuk memudahkan pembaca, kami memberikan tabel yang berisi tentang aspek inti yang dapat mengganggu kualitas sistem jika terjadi ketidakstabilan

Aspek	Deskripsi	Penyebab
Fungsionalitas Fitur Inti	Kegagalan modul inti seperti pendaftaran pasien, mengelola antrian layanan, atau rekam medis yang dapat menghentikan operasional klinik.	Kesalahan logika bisnis sederhana, validasi input yang tidak memadai, atau bug pada alur kerja dasar.
Performa tidak stabil	Terjadi peningkatan respon time yang signifikan sehingga aplikasi berjalan sangat lambat	Query database tidak optimal atau penggunaan sumber daya server yang boros.
Kesalahan UI	UI tidak sesuai, tombol atau form yang tidak berfungsi dengan baik menyebabkan kebingungan pengguna.	Desain UI tidak diuji pada perangkat atau browser target, kode frontend yang tidak responsif

Kerentanan Keamanan	Akses tidak sah ke data rekam medis atau bagian admin karena autentikasi lemah.	Password disimpan sebagai teks biasa dan tidak dienkripsi, tidak ada batasan hak akses.
---------------------	---	---

3.2. Kriteria Kualitas Minimum

Kriteria Kualitas Minimum adalah standar dasar yang harus dipenuhi oleh suatu sistem atau produk perangkat lunak agar dapat dianggap layak digunakan, memenuhi tujuan bisnis, dan memberikan nilai tambah bagi pengguna akhir. Kriteria ini menjadi batas bawah (baseline) yang ditetapkan untuk memastikan bahwa sistem tidak hanya berfungsi, tetapi juga memenuhi aspek keandalan, keamanan, dan kepuasan pengguna yang esensial. Berikut adalah tabel kriteria kualitas minimum pada aplikasi Zoovia

Aspek Kualitas	Kriteria Minimum	Metode Verifikasi
Fungsionalitas Fitur	100% fitur inti berjalan sesuai requirement	Melakukan uji skenario positif/negatif secara manual
<i>Reliability</i>	Uptime $\geq 95\%$ selama 72 jam uji kontinu	Uji ketahanan (endurance testing)
Keamanan	Data sensitif terenkripsi dengan baik dan aman	Akses data dari role tidak dikenal terblokir
Kemudahan Penggunaan	Skor SUS (System Usability Scale) ≥ 70	Kuesioner pengguna akhir

Kriteria ini menjadi fondasi untuk memastikan bahwa aplikasi tidak hanya berjalan, tetapi juga mendukung layanan kesehatan hewan yang responsif, akurat, dan terpercaya. Tanpa pencapaian baseline ini, kami akan kehilangan kepercayaan pengguna dan nilai bisnis sistem menjadi dipertanyakan,

4. Proposed Schedule of Milestones

MILESTONE	DATE
Test Development and Configuration	
Test Plan Complete	25 Februari 2025
Test lab defined (Lingkungan Pengujian)	5 Maret 2025
Test Configured (Persiapan Infrastruktur Pengujian)	15 Maret 2025
Test suite complete (Kumpulan Test case selesai)	30 Maret 2025
Test Execution	
Cycle 1 complete (Pengujian awal/fungsional dasar)	15 April 2025
Cycle 2 complete (Re-testing bug yang salah diperbaiki)	25 April 2025
Cycle 3 complete (Validasi akhir sebelum ke pengguna UAT)	12 Mei 2025

5. Transition

5.1 Entry Criteria

- Semua fitur utama telah dikembangkan dan diuji secara unit testing.
- Rencana pengujian (Test Plan) telah disusun dan disetujui oleh tim QA.
- Lingkungan pengujian (Test Lab) telah dikonfigurasi dan siap digunakan.
- Infrastruktur server dan database telah disiapkan sesuai kebutuhan sistem.
- Semua pengguna utama (admin dan pengguna) telah teridentifikasi untuk UAT.

5.2 Continuation Criteria

- Semua skenario pengujian fungsional dasar telah dijalankan dengan hasil yang memuaskan.
- Perbaikan bug dari pengujian sebelumnya telah dikonfirmasi melalui re-testing.
- Evaluasi performa dan keamanan sistem menunjukkan hasil yang sesuai standar.
- Umpan balik dari pengguna selama UAT telah dikumpulkan dan dianalisis.
- Sistem sudah dalam kondisi stabil tanpa adanya error kritis yang dapat menghambat operasional.

5.3 Exit Criteria

- Sistem telah melewati semua siklus pengujian (Functional, Integration, Performance, Security, dan UAT) tanpa kendala besar.
- Tidak ada bug prioritas tinggi yang masih terbuka.
- Sistem telah diimplementasikan di lingkungan produksi dan diuji dalam kondisi nyata.
- Pengguna telah menerima pelatihan lengkap dan siap mengoperasikan sistem secara mandiri.
- Dokumentasi sistem, termasuk panduan pengguna dan laporan akhir, telah disusun dan diserahkan ke tim operasional.

6. Test Development

Bagian ini menjelaskan bagaimana proses pengujian aplikasi Zoovia dilakukan, termasuk pembuatan dokumen pengujian, metodologi, dan jenis pengujian yang diterapkan.

6.1 Pembuatan dan Pengelolaan Dokumen Pengujian

Tim pengujian bertanggung jawab untuk membuat dan mengelola berbagai dokumen pengujian, yang mencakup:

- 1) Test Cases → Dokumen yang mendeskripsikan skenario uji yang akan dijalankan untuk memeriksa fungsionalitas aplikasi.

Contoh: Menguji apakah fitur pencatatan riwayat kesehatan hewan berfungsi dengan baik.

- 2) Test Plan → Rencana pengujian yang mencakup cakupan pengujian, metode yang digunakan, dan jadwal pelaksanaan.

Contoh: Menentukan tahapan pengujian dari unit testing hingga user acceptance testing (UAT).

- 3) Test Procedures → Langkah-langkah sistematis yang harus dilakukan selama pengujian untuk memastikan hasil yang valid dan konsisten.

Contoh: Langkah-langkah login sebagai dokter hewan dan menginput data pemeriksaan pasien.

- 4) Automated Test Scripts → Kode yang digunakan untuk mengotomatiskan pengujian dengan alat seperti Selenium, Appium, atau Katalon Studio.

Contoh: Menggunakan Selenium untuk menguji kinerja formulir registrasi pasien hewan secara otomatis.

6.2 Metodologi/Strategi Pengujian

Strategi pengujian yang diterapkan dalam aplikasi Zoovia meliputi:

- 1) Black Box Testing → Fokus pada pengujian fungsional tanpa melihat kode sumber. Menguji input dan output sistem.

Contoh: Memastikan fitur pencarian dokter hewan bekerja dengan baik.

- 2) White Box Testing → Pengujian kode sumber untuk mengevaluasi struktur internal aplikasi. Biasanya dilakukan oleh developer.

Contoh: Memeriksa efisiensi kode dalam proses validasi data pengguna.

- 3) Manual Testing → Pengujian dilakukan secara langsung oleh tester tanpa otomatisasi untuk mengevaluasi pengalaman pengguna.

Contoh: Menggunakan aplikasi di berbagai perangkat untuk menguji kemudahan navigasi.

- 4) Automated Testing → Pengujian dengan alat bantu seperti Selenium, Katalon Studio, dan Appium untuk efisiensi dan akurasi lebih tinggi.

Contoh: Mengotomatisasi pengujian input data pasien hewan untuk menghindari kesalahan manusia.

6.3 Jenis Pengujian yang dilakukan

1. Pengujian Fungsional

- 1) Memastikan semua fitur aplikasi **Zoovia** berfungsi sesuai spesifikasi.
- 2) Contoh:
 - a) Pengujian fitur pendaftaran pasien hewan.
 - b) Pengujian jadwal konsultasi dokter hewan.

2. Pengujian UI/UX

- 1) Mengevaluasi tata letak, warna, ikon, dan elemen navigasi untuk memastikan kemudahan penggunaan.
- 2) Contoh:
 - a) Memeriksa apakah tombol navigasi dapat diakses dengan mudah.
 - b) Menilai pengalaman pengguna dalam mengisi formulir konsultasi.

3. Pengujian Performa

- 1) Menguji kecepatan, responsivitas, dan kestabilan aplikasi.
- 2) Contoh:
 - a) Mengukur waktu muat halaman dashboard dokter hewan.
 - b) Menguji apakah aplikasi tetap stabil ketika banyak pengguna melakukan akses bersamaan.

4. Pengujian Keamanan

- 1) Memastikan aplikasi Zoovia terlindungi dari ancaman keamanan.
- 2) Contoh:
 - a) Menguji kekuatan enkripsi data pasien.
 - b) Menguji apakah sistem login rentan terhadap serangan brute force.

5. Pengujian Kompatibilitas

- 1) Memastikan aplikasi dapat berjalan dengan baik di berbagai perangkat dan sistem operasi.
- 2) Contoh:
 - a) Menguji aplikasi pada berbagai versi Android (8.0–12.0) dan Windows 11.
 - b) Memastikan aplikasi tetap responsif di perangkat dengan RAM 2GB, 4GB, dan 8GB serta Laptop dengan RAM 4GB, dan 8GB.

6. Pengujian Integrasi

- 1) Memastikan berbagai komponen aplikasi dapat berfungsi dengan baik bersama-sama.
- 2) Contoh:

- a) Menguji apakah aplikasi bisa menghubungkan sistem Zoovia dengan database pasien hewan.
- b) Menguji integrasi API untuk fitur notifikasi janji temu dokter hewan.

7. Test Configuration and environment

Bagian ini menjelaskan lingkungan pengujian yang digunakan dalam proses pengujian aplikasi Zoovia. Ini mencakup perangkat keras (hardware) dan perangkat lunak (software) yang digunakan oleh tim QA untuk memastikan aplikasi berjalan sesuai spesifikasi. Lingkungan pengujian yang standar dan terdokumentasi dengan baik membantu dalam mendeteksi bug secara konsisten dan memungkinkan replikasi hasil pengujian.

7.1 Perangkat Uji (Test Devices)

Hardware yang digunakan dalam pengujian aplikasi Zoovia terdiri dari berbagai jenis perangkat untuk memastikan aplikasi dapat berjalan dengan baik di berbagai kondisi.

- A. **Perangkat Mobile (Smartphone/Tablet)** digunakan untuk pengujian aplikasi mobile.
- B. **Perangkat Komputer/Laptop** digunakan untuk pengujian aplikasi web.

Digunakan untuk debugging, analisis bug, dan pengelolaan pengujian.

1) Spesifikasi Laptop/PC yang Digunakan:

- Laptop dengan Windows 10 & 11 → Untuk menjalankan alat pengujian.
- RAM 4GB – 8GB → Untuk memastikan kinerja saat menjalankan alat debugging.
- Prosesor Intel Core i5 / i7 atau AMD Ryzen 5 / 7 → Untuk menjalankan emulator dengan lancar.

C. Server & Infrastruktur Backend

Digunakan untuk menyimpan data aplikasi dan menguji komunikasi backend.

- Firebase Firestore → Untuk menyimpan data pengguna dan histori pemeriksaan hewan.
- Firebase Realtime Database → Untuk menguji sinkronisasi data secara real-time.
- Google Cloud Storage → Untuk menyimpan gambar dan dokumen pengguna.

7.2 Software Uji

Software yang digunakan dalam proses pengujian aplikasi Zoovia mencakup alat debugging, pengujian API, otomatisasi pengujian, dan manajemen pengujian.

A. Tools yang Digunakan dalam Pengujian:

1) Pengembangan dan Debugging

- Android Studio → Digunakan untuk debugging, menjalankan emulator, dan analisis performa aplikasi.
- Firebase Crashlytics → Untuk mendeteksi error dan crash dalam aplikasi.

2) Pengujian API & Backend

- Postman → Untuk menguji API aplikasi, memastikan endpoint bekerja dengan benar.
- Firebase Emulator Suite → Untuk menguji fungsi Firebase secara lokal sebelum deployment.

3) Pengujian Otomatis (Automation Testing)

- Selenium → Digunakan untuk menguji antarmuka web.
- Katalon Studio → Alternatif untuk pengujian UI dan API secara otomatis.

4) Manajemen Pengujian & Pelacakan Bug

- Trello → Untuk melacak bug dan mengelola tugas pengujian.
- TestRail → Untuk mengelola test cases dan melacak hasil pengujian.

5) Pengujian Keamanan

- Memastikan data terenkripsi dengan baik dan aman
- Database terpusat di server tertutup

7.3 Konfigurasi Lingkungan Pengujian

Untuk memastikan hasil pengujian yang akurat dan dapat direproduksi, pengujian dilakukan dalam lingkungan yang terkontrol dengan konfigurasi sebagai berikut:

A. Koneksi Internet:

- 1) Wifi Stabil (Minimal 20 Mbps) → Untuk menguji performa aplikasi saat terhubung ke internet.
- 2) Koneksi 4G & 5G → Untuk menguji penggunaan aplikasi di jaringan seluler.

B. Database:

- 1) Firebase Firestore → Untuk pengelolaan data pengguna dan rekam medis hewan.
- 2) SQLite (Lokal) → Untuk pengujian fitur penyimpanan offline.

C. Akun Pengujian:

- 1) Akun Admin → Untuk menguji fitur khusus admin, seperti manajemen data hewan.
- 2) Akun Pengguna → Untuk menguji fitur pemilik hewan dalam aplikasi Zoovia.

D. Skenario Pengujian:

- 1) Mode Offline → Untuk menguji apakah aplikasi dapat menyimpan data saat tidak terhubung ke internet.
- 2) Mode Multi Device → Untuk memastikan data pengguna tetap sinkron di beberapa perangkat.

8. Test Execution

8.1 Resources

Pada tahap eksekusi pengujian, sumber daya yang dibutuhkan meliputi perangkat keras, perangkat lunak, serta sumber daya manusia yang akan terlibat dalam pengujian. Berikut adalah rincian sumber daya yang akan digunakan:

- **Perangkat Keras (Hardware):** Perangkat mobile dan komputer/laptop yang telah disebutkan sebelumnya dalam bagian test configuration, termasuk smartphone dengan berbagai spesifikasi (RAM 2GB, 4GB, 8GB) serta laptop dengan sistem operasi Windows 10 dan 11.
- **Perangkat Lunak (Software):** Android Studio, Firebase, Postman, Selenium, Katalon Studio, Trello, TestRail.
- **Tim Pengujian:** Tim QA akan terdiri dari beberapa tester yang memiliki keahlian dalam pengujian manual dan otomatis. Tim ini juga akan bekerja sama dengan tim pengembang untuk menangani temuan bug yang kritis atau major.

8.2 Test Case and Bug Tracking

Proses pengujian akan didokumentasikan dengan menggunakan Test Case dan Bug Tracking untuk memastikan kualitas dan keberhasilan pelaksanaan pengujian. Test case yang sudah disiapkan sebelumnya akan dieksekusi sesuai dengan jadwal dan prioritas, dan setiap bug yang ditemukan akan dicatat dalam alat pelacakan bug seperti Trello.

- **Test Case Tracking:** Setiap test case yang dieksekusi akan dipantau statusnya melalui TestRail atau alat serupa, dengan status seperti "Passed," "Failed," atau "Blocked."
- **Bug Tracking:** Bug yang ditemukan selama pengujian akan dilaporkan dengan rincian seperti deskripsi, langkah-langkah untuk mereproduksi masalah, tingkat keparahan, dan status pengembangan perbaikan.

8.3 Bug Isolation

Tim QA akan memastikan bahwa setiap bug yang ditemukan dapat diisolasi dengan benar, mengidentifikasi akar masalah untuk perbaikan yang tepat. Bug yang dilaporkan akan dikelompokkan berdasarkan tingkat keparahan (Critical, Major, Minor) dan segera dilaporkan kepada tim pengembang untuk ditangani. Bug yang lebih kompleks mungkin memerlukan penyelidikan lebih lanjut atau diskusi dengan tim pengembang terkait.

8.4 Test Release Management

Pengelolaan versi pengujian sangat penting untuk mengelola perubahan dalam pengujian. Setelah setiap siklus pengujian, versi aplikasi akan diperbarui, dan perubahan yang dilakukan pada aplikasi akan dipastikan sesuai dengan test case yang telah dipersiapkan sebelumnya.

- **Pengujian Berkelanjutan:** Pengujian dilakukan secara bertahap, dengan pembaruan dari siklus sebelumnya akan diuji kembali di siklus selanjutnya.
- **Perilisan Perbaikan:** Setelah temuan bug diatasi, versi terbaru aplikasi akan dirilis untuk pengujian ulang, yang akan difokuskan pada validasi perbaikan.

8.5 Test Cycles

Setiap siklus pengujian berfokus pada aspek yang berbeda, seperti pengujian fungsional dasar, pengujian regresi setelah perbaikan bug, dan validasi akhir sebelum rilis ke pengguna akhir. Setiap siklus pengujian akan mencakup langkah-langkah berikut:

- **Cycle 1:** Pengujian fungsional dasar dan fitur utama aplikasi.
- **Cycle 2:** Pengujian regresi untuk memastikan bug yang diperbaiki tidak memengaruhi fitur lain.
- **Cycle 3:** Pengujian akhir yang lebih mendalam, termasuk validasi performa dan keamanan.

8.6 Test Hours

Jumlah jam pengujian akan dihitung berdasarkan jumlah siklus pengujian yang diperlukan untuk memastikan semua fitur aplikasi diuji dengan benar. Setiap siklus pengujian akan memakan waktu sesuai dengan kompleksitas fitur yang diuji dan temuan bug yang harus ditangani.

9. Risk and Contingency

9.1 Identifikasi Risiko

Dalam proses pengujian aplikasi Zoovia, beberapa risiko mungkin muncul yang dapat mempengaruhi keberhasilan pengujian dan waktu penyelesaian. Beberapa risiko yang dapat muncul termasuk:

- **Bug Kritis di Tahap Akhir Pengujian:** Bug yang ditemukan di tahap akhir pengujian dapat memperlambat proses pengujian dan mempengaruhi jadwal rilis.
- **Keterbatasan Sumber Daya:** Kekurangan sumber daya pengujian, baik manusia maupun alat, dapat menghambat jalannya pengujian yang efisien.
- **Kompatibilitas Perangkat yang Beragam:** Pengujian di banyak perangkat dengan berbagai spesifikasi dapat memunculkan masalah kompatibilitas yang tak terduga.
- **Perubahan Fitur:** Perubahan atau pembaruan fitur di tengah proses pengujian dapat menyebabkan keterlambatan dalam pengujian atau memerlukan pembaruan test case.
- **Keterbatasan Waktu:** Mengingat pengujian dilakukan dalam waktu terbatas, ada risiko bahwa beberapa aspek dari pengujian tidak akan dijalankan dengan cukup mendalam.

9.2 Rencana Kontingensi

Untuk mengatasi risiko tersebut, berikut adalah beberapa langkah mitigasi yang akan diterapkan:

- **Mitigasi Bug Kritis:** Memulai pengujian lebih awal dan melakukan pengujian bertahap akan membantu menemukan bug lebih cepat. Pengujian regresi yang sering juga akan dilakukan untuk memastikan bug perbaikan tidak menyebabkan masalah lain.
- **Mitigasi Keterbatasan Sumber Daya:** Menggunakan alat otomatisasi untuk pengujian tertentu dan melibatkan lebih banyak tester untuk mempercepat pengujian.
- **Mitigasi Kompatibilitas Perangkat:** Pengujian dilakukan di berbagai perangkat dengan berbagai spesifikasi untuk memastikan aplikasi berjalan dengan baik di semua kondisi yang mungkin dihadapi pengguna.

- **Mitigasi Perubahan Fitur:** Setiap perubahan fitur akan memicu pembaruan test case, dan regression testing akan dilakukan untuk memverifikasi pengaruh perubahan terhadap aplikasi secara keseluruhan.
- **Mitigasi Keterbatasan Waktu:** Pengujian akan difokuskan pada fitur-fitur utama yang paling kritis untuk operasi harian, dan bagian-bagian yang lebih kecil akan diuji dalam siklus pengujian lanjutan setelah rilis pertama.

10. Change History

Version	Change Date	By	Description
1.0	2025-03-08	Team Zoovia	New Document

11. Reference document

Huda, B. and Lia Hananto, A. (2023) 'Penerapan Software Testing Life Cycle Pada Pengujian Otomatisasi Platform Dzikra', CSRID Journal, 15(1), pp. 1–11. Available at: <https://www.doi.org/10.22303/csrid.15.1.2023.01-11>.

Sistem Informasi Manajemen Pembuatan Perangkat Lunak "Sistem Rental Mobil Berbasis Online", H., Industri, F.T. and Indonesia, U.I. (2022) 'Pengujian Otomatis Aplikasi Mobile Dengan Teknik Black-Box Menggunakan Appium Pada Aplikasi Otomatis Aplikasi Mobile Dengan Teknik Black-Box Menggunakan Appium'.

12. FAQ

1. Umum

Q: Apa tujuan utama dari dokumen Test Plan ini?

A: Dokumen Test Plan dibuat untuk memberikan panduan lengkap tentang strategi, cakupan, metode, serta eksekusi pengujian perangkat lunak guna memastikan bahwa aplikasi Zoovia berfungsi sesuai spesifikasi dan bebas dari bug sebelum dirilis.

Q: Siapa yang bertanggung jawab dalam pengujian ini?

A: Tim QA (Quality Assurance), tester, serta beberapa anggota tim pengembang bertanggung jawab dalam pengujian aplikasi. Setiap bug yang ditemukan akan dilaporkan kepada tim pengembang untuk diperbaiki.

Q: Apa perbedaan antara Test Plan dan Test Case?

A:

- Test Plan: Dokumen strategis yang mencakup cakupan, jadwal, dan metodologi pengujian.
- Test Case: Skenario pengujian spesifik yang mendeskripsikan langkah-langkah untuk memverifikasi fitur aplikasi.

Q: Apa perbedaan antara pengujian manual dan otomatis?

A:

- Pengujian Manual: Dilakukan oleh tester secara langsung tanpa menggunakan alat otomatisasi. Cocok untuk UI/UX testing dan exploratory testing.
- Pengujian Otomatis: Menggunakan skrip atau alat khusus untuk menjalankan test case secara otomatis, cocok untuk regresi dan performa testing.

2. Proses Pengujian

Q: Kapan pengujian dimulai dan kapan dianggap selesai?

A:

- Pengujian dimulai setelah tahap pengembangan fitur utama selesai dan aplikasi dapat digunakan.
- Pengujian dianggap selesai jika semua test case telah diuji dan tidak ada bug kritis yang tersisa.

Q: Apa saja jenis pengujian yang dilakukan dalam proyek ini?

A: Jenis pengujian yang dilakukan meliputi:

- Functional Testing → Memastikan fitur aplikasi bekerja dengan baik.
- UI/UX Testing → Menilai kenyamanan dan tampilan aplikasi.
- Performance Testing → Menguji kecepatan dan stabilitas aplikasi.
- Security Testing → Memeriksa kerentanan aplikasi terhadap serangan.
- Compatibility Testing → Memastikan aplikasi berjalan di berbagai perangkat.

Q: Apa yang terjadi jika ditemukan bug saat pengujian?

A: Jika ditemukan bug:

- Bug dicatat dalam bug tracking tool (misalnya Trello).
- Bug dikategorikan berdasarkan tingkat keparahannya (Critical, Major, Minor).
- Bug dikirim ke developer untuk diperbaiki.
- Setelah diperbaiki, dilakukan regression testing untuk memastikan perbaikan tidak menyebabkan bug baru.

Q: Bagaimana cara menentukan apakah bug itu kritis atau tidak?

A: Bug dikategorikan berdasarkan dampaknya terhadap aplikasi:

- Kritis: Aplikasi crash, gagal login, atau fitur utama tidak berfungsi.
- Mayor: Fitur sekunder tidak bekerja dengan baik.
- Minor: Masalah kecil seperti tampilan UI yang salah.
- Trivial: Kesalahan ejaan atau perubahan kecil lainnya.

Q: Bagaimana cara memastikan bahwa tidak ada fitur yang terlewat saat pengujian?

A: Setiap fitur harus diuji berdasarkan test case yang telah dibuat sebelumnya. Test case mencakup skenario-skenario berbeda yang harus diuji untuk memastikan aplikasi berjalan dengan baik di berbagai kondisi.

3. Lingkungan Pengujian

Q: Apa saja spesifikasi perangkat yang digunakan untuk pengujian?

A: Pengujian dilakukan pada berbagai perangkat, di antaranya:

Hardware:

- Android 8.0 hingga Android 12.0
- Windows 11

Software:

- Smartphone dengan RAM 2GB, 4GB, dan 8GB
- Laptop dengan RAM 4GB, dan 8GB
- Katalon Studio/Selenium
- Web Server (Laravel)
- Database Server (SQLite)
- Browser (Chrome, Firefox, Safari, Edge).

Q: Apakah pengujian dilakukan pada emulator atau perangkat asli?

A: Pengujian dilakukan di keduanya:

- Emulator digunakan untuk debugging cepat dan pengujian awal.
- Perangkat asli digunakan untuk pengujian kompatibilitas, UI, dan performa.

Q: Apa saja software yang digunakan dalam pengujian ini?

A:

- Android Studio → Untuk debugging dan pengujian emulator.
- Firebase → Untuk mengelola backend data.
- Postman → Untuk Menguji API.
- Trello → Untuk tracking bug dan progress pengujian.
- Katalon/Selenium → Otomatisasi pengujian web (UI & fungsionalitas).

4. Dokumentasi dan Pelaporan

Q: Bagaimana cara melaporkan bug yang ditemukan?

A: Bug dicatat dalam bug tracking system dengan format berikut:

- ID Bug: Nomor unik untuk bug.
- Deskripsi: Penjelasan singkat tentang bug.
- Langkah Reproduksi: Cara mengulang bug tersebut.
- Tingkat Keparahan: Kritis, Major, Minor, atau Trivial.
- Screenshot/Log Error: Bukti visual atau log dari error.
- Status: Open, In Progress, Resolved, Closed.

Q: Bagaimana pengujian terdokumentasi?

A: Semua hasil pengujian, termasuk bug yang ditemukan, test case yang dijalankan, dan status pengujian, dicatat dalam Test Report yang dibuat setelah pengujian selesai.

5. Risiko dan Solusi

Q: Apa risiko terbesar dalam proses pengujian ini?

A: Beberapa risiko terbesar termasuk:

- Banyak bug ditemukan di tahap akhir → Solusi: Mulai pengujian lebih awal.
- Aplikasi tidak stabil di beberapa perangkat → Solusi: Lakukan compatibility testing lebih sering.
- Kurangnya sumber daya pengujian (tester terbatas) → Solusi: Gunakan automated testing untuk beberapa test case.

Q: Bagaimana jika ada perubahan fitur saat pengujian berlangsung?

A: Jika ada perubahan fitur:

- Test case diperbarui agar sesuai dengan perubahan tersebut.
- Regression testing dilakukan untuk memastikan fitur lain tidak terganggu.

Q: Bagaimana jika ada fitur yang gagal saat tahap akhir pengujian?

A: Jika fitur gagal saat tahap akhir:

- Jika fitur krusial, pengujian diperpanjang dan bug harus diperbaiki sebelum rilis.
- Jika fitur minor, bisa ditunda untuk diperbaiki dalam update berikutnya.

6. Penyelesaian dan Penutupan Pengujian

Q: Kapan pengujian dianggap selesai?

A: Pengujian dianggap selesai jika:

- Semua test case telah diuji.
- Tidak ada bug kritis atau major yang tersisa.
- Performa dan keamanan aplikasi memenuhi standar yang ditetapkan.

Q: Apa langkah selanjutnya setelah pengujian selesai?

A: Setelah pengujian selesai:

- Test Report disusun dan dikirim ke stakeholder.
- Aplikasi siap dirilis atau diperbaiki lebih lanjut.
- Jika perlu, tim QA akan melakukan pengujian tambahan setelah rilis (post-release testing).

7. Tambahan

Q: Bagaimana jika ditemukan bug setelah aplikasi dirilis?

A: Jika bug ditemukan setelah rilis, tim akan:

- Mengumpulkan laporan bug dari pengguna.
- Menentukan tingkat keparahan bug.
- Merilis update atau patch secepat mungkin untuk memperbaiki bug tersebut.

Q: Bagaimana cara memastikan bahwa aplikasi tetap berkualitas setelah rilis?

A:

- Melakukan monitoring bug menggunakan tools seperti Firebase Crashlytics.
- Mengumpulkan feedback dari pengguna dan melakukan update berkala.
- Melakukan pengujian tambahan setelah rilis jika diperlukan.